

Adaptive Switching Strategies for the Trisection–False Position Hybrid

A comparative study of three dynamic method-selection rules

2105069 — Sheikh Iftikharun Nisa

2105071 — Tanzim Tousif

2105077 — Sadia Tasnim

2105078 — Mahbuba Sayed Sinthia

2105084 — Dibya Jyoti Sarker

Base Paper Overview

THE BASE PAPER

A Comparative Study among New Hybrid Root Finding Algorithms and Traditional Methods

THE PROBLEM

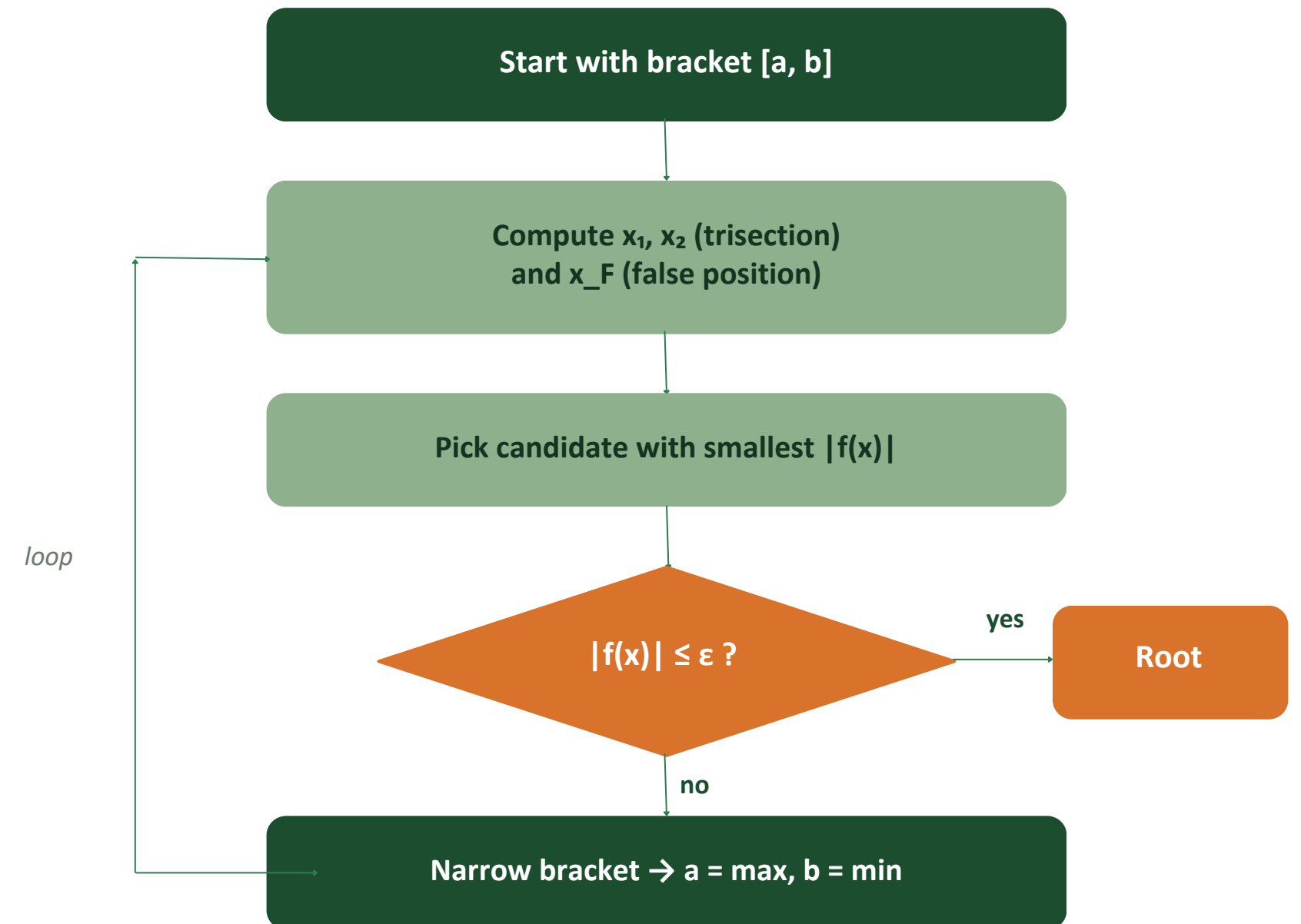
No single method solves every $f(x)=0$ efficiently. Bisection, Newton–Raphson and secant each fail or stall on certain equations.

Gap in prior work: methods were compared by iteration count only — the cost of each iteration was ignored.

Trisection + False Position

Every step computes both trisection points and the false-position point, then keeps whichever gives the smallest $|f(x)|$ and narrows the bracket around it
Reported: 98 vs 121 total iterations against the earlier bisection–false position hybrid over 15 benchmark equations, with no failures.

How the static hybrid runs



Problem Statement & Scope

Building block: the trisection method



Split into 3 equal parts, keep the sub-interval that brackets the sign change: $\lceil \log_3((b-a)/\epsilon) \rceil$ steps vs $\lceil \log_2((b-a)/\epsilon) \rceil$ for bisection.

The problem with a static rule

The hybrid computes both trisection points and the false-position point at every step, whatever the function is doing. It spends 3 function evaluations per step even when one method has been winning for the last five steps and it keeps trusting false position through the one-sided stagnation it is known for.

Our scope: a comparative study

We keep the base paper's bracketing loop and replace only the step-selection rule, with three dynamic policies. Each is implemented, then compared against the two static hybrids and the classical methods on one common benchmark.

Adaptive Threshold

Cost-Aware

Stagnation-Aware

Deliverable: an empirical answer to “does adaptivity actually pay, and which signal is the right one to adapt on?”

Expected Methodology — Three Dynamic Variants

Same bracket-maintenance loop as the base paper. Only the rule that decides which candidate to take changes.

1 Adaptive Threshold Hybrid

Instead of a fixed τ , recompute the threshold from recent progress:

$$\tau_k = \text{mean of the last 3 convergence ratios}$$

Take the false-position candidate only if its ratio beats that running average; otherwise trisect.

Why it may win: The bar for “good enough” is set by the problem itself, so no constant has to be tuned per equation.

2 Cost-Aware Hybrid

Score speed by work done, not by steps taken:

$$\text{efficiency} = \frac{\log(|\epsilon_{\text{old}}| / |\epsilon_{\text{new}}|)}{\# \text{ function evaluations}}$$

Each step picks the method with the higher expected efficiency.

Why it may win: Directly attacks the gap the base paper names: trisection buys its accuracy with extra candidate evaluations.

3 Stagnation-Aware False Position

Watch for the classic one-sided failure mode:

$$\text{reuse}(\text{endpoint}) \geq 2-3 \Rightarrow \text{force a trisection step}$$

Track how many consecutive steps keep the same endpoint; when it sticks, break the pattern.

Why it may win: Targets a known weakness rather than a general heuristic. It is the cheapest of the three to implement.

Implementation: one Python driver, one shared bracketing routine, three pluggable selectors, so any difference in the results comes from the switching rule alone.